

Nanggroe OS AI Blueprint Package

This document is organized as multiple markdown files in one workspace-friendly package. Each section is written so it can be split into separate files later without rewriting the content.

1) PROMPT .md

Nanggroe OS AI — Master System Prompt

You are Nanggroe OS AI, a modular autonomous robotics operating system and agent framework.

Your mission is to transform a human goal into a working real-world system by:

- detecting available hardware,
- identifying missing components,
- suggesting compatible firmware and libraries,
- generating mission plans,
- building dashboards and control flows,
- providing calibration guidance,
- logging activities and learning from prior sessions,
- syncing local and online memory,
- operating across multiple domains such as drone, rover, marine, agriculture, and industrial automation.

Core Principles

1. Prefer safety, reliability, and recoverability.
2. Keep real-time control deterministic.
3. Use AI for planning, reasoning, recommendation, and orchestration, not for low-level flight stabilization.
4. Support offline-first operation.
5. Save session state, hardware state, mission state, and calibration state.
6. Detect hardware automatically when possible.
7. Ask for missing details only when they are truly necessary.
8. Suggest the simplest viable architecture before advanced features.
9. Treat every project as a modular system with plugins and adapters.
10. When multiple solutions exist, explain tradeoffs and choose the most practical default.

Supported System Types

- Drone / UAV
- Rover / UGV
- Marine / Boat
- Agriculture / Smart Farming
- Industrial Automation / Factory
- Sensor Network / IoT
- CNC / Motion Control
- Monitoring / Telemetry / HMI

Required Output Format

For every request, respond with:

1. Project summary
2. Detected or assumed hardware
3. Missing components
4. System architecture
5. Wiring / interface plan
6. Firmware / software plan
7. AI agent plan
8. Dashboard / control plan
9. Calibration plan
10. Logging / memory plan
11. Risks and constraints
12. Next actions

Operating Modes

Mode A — Discovery

Used when hardware is unknown. Detect, infer, and propose.

Mode B — Planning

Used when the concept is known. Produce architecture, modules, and interfaces.

Mode C — Build

Used when implementation begins. Produce tasks, file structure, and code scaffolds.

Mode D — Debug

Used when the system is failing. Diagnose symptoms, isolate causes, and recommend tests.

Mode E — Optimize

Used when the system already works. Reduce cost, weight, power, latency, and complexity.

Agent Roles

Hermes

Strategic reasoning agent.

Responsibilities:

- interpret natural language goals,
- create missions,
- summarize telemetry,
- recommend next steps,
- coordinate higher-level decisions.

PicoClaw

Tactical real-time assistant.

Responsibilities:

- sensor reactions,
- obstacle response,
- immediate warnings,
- local control decisions,
- short-horizon adjustments.

Hard Rules

- Do not rely on AI for safety-critical motor stabilization.
 - Do not suggest a complex solution when a simpler one is enough.
 - Do not hide uncertainty.
 - Do not claim features that are not supported by the current hardware.
 - Always identify what is missing before promising autonomy.
-

2) STRUCTURE.md

Nangroe OS AI — System Structure

1. Purpose

Nangroe OS AI is an autonomous robotics operating system for hardware detection, mission orchestration, telemetry, AI-assisted control, offline memory, and cloud synchronization.

2. Primary Goals

- Turn natural language into executable missions.
- Detect and profile connected hardware.
- Suggest missing hardware and software.
- Generate control logic, dashboards, and logs.
- Support modular device classes.
- Learn from prior sessions and improve recommendations.

3. System Layers

3.1 User Interface Layer

- Web dashboard

- Android control interface
- Telegram bot interface
- Voice input/output interface

3.2 Agent Layer

- Hermes: strategic planning and reasoning
- PicoClaw: tactical and real-time reaction support

3.3 Orchestration Layer

- mission planning
- device configuration
- state management
- calibration workflows
- logging and session memory

3.4 Hardware Abstraction Layer

- unified interface over different boards and peripherals
- adapter modules for drone, rover, boat, and factory devices

3.5 Device Control Layer

- board-specific firmware adapters
- actuator control
- sensor acquisition
- communication bridges

3.6 Telemetry Layer

- GPS
- battery
- altitude
- signal strength
- mission state
- sensor health

3.7 Memory Layer

- local session memory
- project memory
- calibration memory
- online sync memory

3.8 Plugin Layer

- drone plugin
- agriculture plugin
- mapping plugin
- payload plugin

- factory plugin
- marine plugin
- rover plugin

4. Core Data Objects

4.1 Device

Represents a detected hardware unit.

Fields:

- id
- type
- board
- firmware
- port
- health
- capabilities

4.2 Mission

Represents a user goal.

Fields:

- id
- objective
- mode
- constraints
- route
- payload
- status

4.3 Session

Represents one active work context.

Fields:

- id
- timestamp
- device context
- mission context
- logs
- calibration data

4.4 Agent State

Represents Hermes or PicoClaw state.

Fields:

- active task
- confidence
- memory references
- recent observations

5. Operating Philosophy

- Offline-first
 - Modular by design
 - Simple before advanced
 - Deterministic control at the edge
 - AI for planning, not for unsafe direct actuation
 - Always store history and lessons learned
-

3) DESIGN.md

Nangroe OS AI — Design Document

1. Design Objectives

- Make complex robotics hardware easy to configure.
- Allow prompt-based task creation.
- Auto-detect hardware whenever possible.
- Produce useful outputs even when some hardware is missing.
- Support future expansion from one device class to many.

2. Design Constraints

- Real-time actuation must remain deterministic.
- Hardware diversity must be handled by adapters.
- AI must not be the only source of truth for safety-critical control.
- The system must work offline with optional online sync.
- The first version must be small enough to finish.

3. Architectural Decisions

3.1 Use an OS-style core

Reason: the product is not just a drone controller. It is a general robotics orchestration platform.

3.2 Separate strategy from execution

- Hermes decides and plans.
- PicoClaw reacts and assists.
- Firmware executes deterministically.

3.3 Use plugin architecture

Each hardware category should be a plugin so the system can grow without rewriting the core.

3.4 Use a hardware abstraction layer

The same mission language should map to different physical devices through adapters.

4. Reference Stack

Runtime

- Linux
- Python service layer
- Web dashboard frontend

Robotics / Device Control

- MAVLink for aerial systems
- MQTT for messaging
- Serial / USB / CAN / I2C / SPI adapters

AI Layer

- local lightweight model for command understanding
- optional larger model for planning
- vision models for detection tasks

Storage

- SQLite for local state
- optional cloud database for sync

5. Agent Design

Hermes

Primary planner and analyst.

Responsibilities:

- interpret prompts
- generate missions
- recommend hardware
- summarize telemetry
- write logs and lessons

PicoClaw

Realtime helper.

Responsibilities:

- observe sensor streams
- flag hazards
- suggest immediate actions
- assist with obstacle logic

6. UX Design Goals

- User should describe a goal in natural language.
- The system should answer with a structured plan.
- The system should highlight missing parts.
- The dashboard should show current state without clutter.

- Telegram and Android should be lightweight control surfaces.

7. Drone Use Case Design

For the drone module:

- detect flight hardware
- check GPS, battery, camera, payload, and communication modules
- generate mapping or delivery mission
- return-to-home on signal loss or low battery
- store mission logs locally
- sync when online

8. Mapping Use Case Design

For agriculture and survey:

- generate flight grid
- capture images automatically
- tag images with GPS
- estimate area
- calculate mission result
- create a report and save it

9. Learning Design

The system should improve from:

- repeated hardware detection results
- calibration outcomes
- failed missions
- successful missions
- user corrections

10. Non-Goals for V1

- no custom flight controller from scratch
- no anti-clone or advanced security work in the first build
- no swarm systems in the first build
- no overly complex autonomy before the core pipeline works

4) ROADMAP.md

Nanggroe OS AI — Roadmap

Phase 0 — Definition

- finalize vision
- finalize core modules

- finalize target use case
- finalize naming

Phase 1 — Core OS Skeleton

- project structure
- logging system
- session memory
- configuration loader
- plugin loader
- dashboard shell

Phase 2 — Hardware Detection

- board detection
- port detection
- sensor enumeration
- capability detection
- missing component suggestions

Phase 3 — Mission Engine

- prompt parsing
- mission generation
- task breakdown
- mission state machine
- mission history

Phase 4 — Telemetry Engine

- battery state
- GPS state
- signal state
- sensor health
- mission status

Phase 5 — Hermes Assistant

- natural language interaction
- structured recommendations
- troubleshooting help
- summaries and reports

Phase 6 — Drone Vertical

- basic autopilot integration
- waypoint missions
- return home
- mapping capture
- payload release

Phase 7 — Android and Telegram

- Android web dashboard support
- Telegram command bridge
- notifications
- remote status reporting

Phase 8 — Voice Interface

- speech-to-text
- text-to-speech
- voice command handling

Phase 9 — Multi-Agent Expansion

- Hermes strategic agent
- PicoClaw tactical agent
- more specialized agents later

Phase 10 — Cross-Domain Expansion

- rover
- marine
- agriculture
- factory
- CNC

Rule

Do not move to the next phase until the current phase works reliably.

5) MODULES.md

Nanggroe OS AI — Modules

Core Modules

1. core

System boot, configuration, lifecycle, state.

2. hardware

Detection, profiling, compatibility, abstraction.

3. agents

Hermes and PicoClaw orchestration.

4. telemetry

Live sensor reading and health monitoring.

5. missions

Mission planning, execution, result tracking.

6. mapping

Grid missions, image capture, geotagging, area measurement.

7. vision

Object detection, face tracking, obstacle detection.

8. voice

STT, TTS, audio prompts, alerts.

9. memory

Local logs, calibration history, project memory.

10. plugins

Domain-specific extensions.

11. dashboard

UI, charts, map, device manager, mission controls.

12. firmware

Firmware templates, upload helpers, config files.

Suggested Plugin Types

- air
- ground
- marine
- agri
- factory
- mapping
- payload
- voice

6) INTERFACES.md

Nanggroe OS AI — Interfaces

1. Human Interface

Inputs

- text prompt
- voice prompt
- dashboard controls
- Telegram commands
- Android controls

Outputs

- structured plan
- warnings
- telemetry
- map results
- logs
- spoken feedback

2. Hardware Interface

Discovery Inputs

- USB
- serial
- CAN
- I2C
- SPI
- GPS
- camera
- sensors

Hardware Outputs

- servo
- relay
- motor
- buzzer
- light
- payload release

3. Software Interface

Internal APIs

- detect_device()
- build_mission()
- start_mission()
- stop_mission()
- return_home()
- capture_map()
- sync_memory()

- calibrate_sensor()

4. Messaging Interface

Supported channels

- MAVLink
- MQTT
- WebSocket
- serial bridge
- Telegram bot

5. Storage Interface

Local

- SQLite
- JSON
- log files

Online

- synced project storage
- telemetry mirror
- mission archive

7) ARCHITECTURE.md

Nanggrove OS AI — Reference Architecture

Layer 1: Presentation

- Web dashboard
- Android browser dashboard
- Telegram bot
- voice interface

Layer 2: Orchestration

- mission engine
- agent engine
- plugin manager
- memory manager

Layer 3: Intelligence

- Hermes strategic reasoning
- PicoClaw tactical support
- optional local LLM

- vision models

Layer 4: Device Abstraction

- hardware profiles
- adapters
- driver bridges
- config translators

Layer 5: Device Control

- firmware communication
- telemetry I/O
- actuator control
- sensor collection

Layer 6: Data

- logs
- mission history
- calibration data
- sync queue

Guiding Rule

If a function belongs at a lower layer, do not implement it higher up.

8) FILES.md

Suggested Repository Layout

```
```text
```

```
NanggroeeOS/
```

```
|— README.md
|— PROMPT.md
|— STRUCTURE.md
|— DESIGN.md
|— ROADMAP.md
|— MODULES.md
|— INTERFACES.md
|— ARCHITECTURE.md
|— core/
| |— bootstrap
| |— config
| |— state
```

```
| └─ lifecycle
| └─ agents/
| └─ hermes/
| └─ picoclaw/
| └─ hardware/
| └─ detection/
| └─ profiles/
| └─ adapters/
| └─ telemetry/
| └─ missions/
| └─ mapping/
| └─ vision/
| └─ voice/
| └─ memory/
| └─ dashboard/
| └─ plugins/
| └─ firmware/
| └─ logs/
| └─ docs/
```

---

# 9) `README.md`

```md

Nanggrove OS AI

Nanggrove OS AI is a modular autonomous robotics operating system for real-world machines.

It is designed to:

- detect hardware,
- plan missions,
- assist with control,
- store memory,
- sync online when available,
- support drones, rovers, marine systems, agriculture, and industrial automation.

Main Agents

- Hermes: strategic AI planner
- PicoClaw: realtime tactical assistant

Main Surfaces

- web dashboard

- Android control
- Telegram bot
- voice commands

First Vertical

Start with autonomous drone mapping and payload workflows.

Principle

Build the OS first, then extend into hardware platforms.

10) NEXT_STEPS.md

Next Steps

1. Confirm the first vertical use case.
 2. Lock the MVP scope.
 3. Create the core folder structure.
 4. Build the dashboard shell.
 5. Build hardware detection.
 6. Build the mission engine.
 7. Build telemetry logging.
 8. Add Hermes assistant responses.
 9. Connect Telegram bridge.
 10. Connect one real device type first.
-

11) GLOSSARY.md

Glossary

Hermes

Strategic AI agent.

PicoClaw

Realtime tactical AI assistant.

Mission

A goal expressed in structured operational terms.

Hardware Profile

A detected or declared set of connected devices and capabilities.

Telemetry

Live operational data from hardware.

Plugin

A module that extends the OS for a specific domain.

Abstraction Layer

A layer that hides hardware-specific details behind a unified interface.

12) SUMMARY.md

Nanggrove OS AI — Summary

Nanggrove OS AI is a modular autonomous robotics operating system built around:

- hardware detection,
- AI orchestration,
- mission planning,
- telemetry,
- memory,
- voice,
- dashboard control,
- and plugin-based expansion.

The first product vertical should be a practical autonomous drone mapping workflow.

SPLIT NOTE

This package is intentionally written so each section can be copied into its own `.md` file with minimal editing.

13) EXTRA FILES FOR FULL PROJECT PACK

`MVP\_SPEC.md`

```md

## # Nanggrove OS AI — MVP Specification

### ## MVP Objective

Build the smallest useful version of Nanggrove OS AI that can:

- detect hardware,
- manage missions,
- show telemetry,
- support Hermes reasoning,
- sync logs,
- control at least one real device class.

### ## MVP Scope

#### #### Included

- project bootstrap
- hardware detection
- telemetry reading
- mission creation
- dashboard shell
- Telegram bridge
- local memory
- basic voice output
- one device plugin

#### #### Excluded

- swarm systems
- anti-clone security
- full multi-domain support
- custom flight controller
- complex autonomy beyond the first use case

### ## Success Criteria

- user can define a mission in natural language
- system can translate mission into structured tasks
- system can store and retrieve session state
- system can display live or simulated telemetry
- system can connect to one device/plugin reliably

## DATA\_MODEL.md

## # Nanggrove OS AI — Data Model

### ## Entities

#### ### Project

- id
- name
- description
- created\_at
- updated\_at
- tags

#### ### Session

- id
- project\_id
- mode
- timestamp
- notes
- logs

#### ### Device

- id
- session\_id
- device\_type
- board
- firmware
- port
- capabilities
- health

#### ### Mission

- id
- project\_id
- objective
- status
- priority
- route
- payload
- constraints

#### ### TelemetryPoint

- id
- device\_id
- timestamp
- latitude
- longitude
- altitude
- battery

- signal
- sensor\_payload

#### ### CalibrationProfile

- id
- device\_id
- name
- values\_json
- version
- created\_at

#### ### AgentMessage

- id
- agent\_name
- session\_id
- role
- content
- confidence
- timestamp

#### ### MemoryItem

- id
- key
- value
- namespace
- sensitivity
- created\_at

## API.md

### # Nangroe OS AI — API Sketch

#### ## Core Operations

##### ### `detect\_device()`

Detect connected hardware and return a structured profile.

##### ### `build\_mission(prompt)`

Convert a natural language goal into a mission object.

##### ### `start\_mission(mission\_id)`

Start a mission and stream status.

##### ### `stop\_mission(mission\_id)`

Stop the active mission safely.

#### `return\_home(device\_id)`

Request autonomous return-to-home if supported.

#### `sync\_memory()`

Sync local memory to online storage.

#### `save\_log(entry)`

Append a log entry to the current session.

#### `get\_telemetry(device\_id)`

Fetch latest telemetry snapshot.

#### `calibrate(device\_id, profile)`

Apply or update calibration settings.

## API Style

- JSON request/response
- deterministic status codes
- explicit error messages
- no hidden state changes

## CONFIG.md

# Nangroe OS AI — Configuration

## Config Goals

- easy to edit
- versioned
- device-aware
- safe by default

## Suggested Config Sections

#### `core`

- app name
- environment
- log level
- sync policy

#### `agents`

- Hermes model
- PicoClaw model

- thresholds
- memory limits

#### ### `hardware`

- board profiles
- supported protocols
- default device plugin

#### ### `missions`

- default safety rules
- mission timeout
- return-home policy

#### ### `dashboard`

- theme
- map provider
- telemetry refresh rate

#### ### `voice`

- stt engine
- tts engine
- wake word

#### ### `storage`

- local database path
- cloud endpoint
- cache limits

## ADAPTERS.md

### # Nangroe OS AI — Adapters

#### ## Adapter Purpose

Adapters translate generic OS commands into hardware-specific actions.

#### ## Adapter Types

##### ### Drone Adapter

Maps missions to flight commands, camera capture, and payload actions.

##### ### Rover Adapter

Maps missions to wheel movement, obstacle handling, and ground navigation.

##### ### Marine Adapter

Maps missions to water navigation and floating recovery logic.

#### ### Agriculture Adapter

Maps missions to mapping, spraying, and drop operations.

#### ### Factory Adapter

Maps missions to conveyor, actuator, and process automation logic.

### ## Adapter Contract

Each adapter should support:

- initialize()
- detect()
- arm()
- execute()
- pause()
- stop()
- report\_status()

## HARDWARE\_DETECTION.md

# Nangroe OS AI — Hardware Detection

### ## Detection Goals

- detect connected boards
- detect ports
- detect sensors and actuators
- infer missing parts
- estimate compatibility

### ## Detection Sources

- USB VID/PID
- serial banners
- I2C scan
- SPI probe
- GPS lock
- camera handshake
- telemetry heartbeat

### ## Output Format

- detected devices
- confidence score
- supported actions
- missing components

- warnings
- next steps

## DASHBOARD\_SPEC.md

# Nanggroe OS AI — Dashboard Spec

### ## Dashboard Sections

- overview
- telemetry
- maps
- missions
- devices
- memory
- logs
- voice
- plugins

### ## Dashboard Behaviors

- responsive on Android
- clear status colors
- simple controls for non-technical use
- advanced panels hidden by default

### ## Core Widgets

- live map
- battery card
- GPS card
- mission timeline
- device health panel
- AI chat panel
- alert feed

## TELEGRAM\_SPEC.md

# Nanggroe OS AI — Telegram Spec

### ## Commands

- `/status`
- `/map`
- `/mission`
- `/return`



- `/photo`
- `/logs`
- `/calibrate`
- `/help`

### ## Telegram Role

- quick status access
- emergency command channel
- lightweight notifications
- mission summary delivery

### ## Safety

- critical commands require confirmation when needed
- all actions must be logged

## VOICE\_SPEC.md

### # Nanggroe OS AI — Voice Spec

#### ## Input

- wake word
- push-to-talk
- optional offline voice recognition

#### ## Output

- short warnings
- mission confirmations
- telemetry summaries
- safety alerts

#### ## Voice Tone

- short
- direct
- low ambiguity
- action-oriented

## MAPPING\_SPEC.md

### # Nanggroe OS AI — Mapping Spec

#### ## Mapping Goals

- auto route generation

- image capture
- geotagging
- area calculation
- mission summary

### ## Mapping Pipeline

1. define target area
2. create survey grid
3. fly or move through grid
4. capture data
5. store locally
6. generate map output
7. sync when online

## LOGGING\_SPEC.md

### # Nanggroe OS AI — Logging Spec

#### ## Log Types

- mission logs
- telemetry logs
- calibration logs
- error logs
- agent logs
- user command logs

#### ## Log Rules

- timestamp everything
- include session id
- include device id
- separate warnings from errors
- keep logs queryable

## TEST\_PLAN.md

### # Nanggroe OS AI — Test Plan

#### ## Test Levels

##### ### Unit Tests

- device parsing
- mission parsing
- config loading

#### ### Integration Tests

- dashboard to backend
- Telegram to mission engine
- telemetry to memory

#### ### Hardware-in-the-Loop

- one board connected
- one sensor connected
- one actuator connected

#### ### System Tests

- full prompt to mission flow
- offline operation
- sync recovery

#### ## Exit Criteria

- no critical crashes
- predictable output
- saved logs
- successful restore after restart

## RELEASE\_PLAN.md

### # Nanggroe OS AI — Release Plan

#### ## Release 0.1

- core structure
- logging
- config
- dashboard shell

#### ## Release 0.2

- hardware detection
- telemetry
- mission model

#### ## Release 0.3

- Hermes agent
- Telegram bridge
- voice feedback

#### ## Release 0.4

- first device plugin
- first mapping workflow
- first autonomous loop

## Release Rule

Only ship when the system is stable enough to be reused.

---

## 14) FULL REPOSITORY READY PACK

### REPO\_STRUCTURE.md

# Nanggro OS AI — Repository Structure

```text

Nanggro OS/

```
|
|— README.md
|— LICENSE
|— .gitignore
|— docker-compose.yml
|— requirements.txt
|— package.json
|— pyproject.toml
|— .env.example
|
|— docs/
|   |— PROMPT.md
|   |— STRUCTURE.md
|   |— DESIGN.md
|   |— ROADMAP.md
|   |— MODULES.md
|   |— INTERFACES.md
|   |— ARCHITECTURE.md
|   |— MVP_SPEC.md
|   |— DATA_MODEL.md
|   |— API.md
|   |— CONFIG.md
|   |— ADAPTERS.md
|   |— HARDWARE_DETECTION.md
|   |— DASHBOARD_SPEC.md
```

- TELEGRAM_SPEC.md
- VOICE_SPEC.md
- MAPPING_SPEC.md
- LOGGING_SPEC.md
- TEST_PLAN.md
- RELEASE_PLAN.md
- GLOSSARY.md

- backend/

- app/

- main.py
 - config/
 - core/
 - agents/
 - telemetry/
 - hardware/
 - missions/
 - mapping/
 - voice/
 - vision/
 - plugins/
 - dashboard/
 - memory/
 - firmware/
 - adapters/
 - api/
 - logs/
 - tests/

- requirements.txt

- frontend/

- public/

- src/

- pages/
 - components/
 - widgets/
 - layouts/
 - maps/
 - telemetry/
 - ai/
 - missions/
 - styles/

```
├── package.json
├── firmware/
│   ├── esp32/
│   ├── arduino/
│   ├── stm32/
│   ├── pixhawk/
│   └── examples/
├── plugins/
│   ├── air/
│   ├── ground/
│   ├── marine/
│   ├── agri/
│   ├── mapping/
│   ├── payload/
│   └── factory/
├── telemetry/
├── logs/
├── configs/
├── datasets/
├── simulations/
├── scripts/
├── tools/
├── docker/
└── assets/
```

`README\_REPO.md`

``md

Nanggrove OS AI

Vision

Nanggrove OS AI is a modular autonomous robotics operating system focused on:

- hardware detection,
- AI orchestration,
- mission planning,
- telemetry,
- autonomous workflows,
- memory and synchronization.

Main Goal

Transform complex robotics workflows into prompt-driven operational systems.

First Vertical

Autonomous drone mapping and payload workflows.

Main Agents

Hermes

Strategic AI orchestration.

PicoClaw

Realtime tactical assistant.

Features

- hardware auto-detection
- telemetry dashboard
- mission planner
- AI copilot
- Android dashboard
- Telegram integration
- voice feedback
- offline-first memory
- plugin architecture

Stack

Backend

- Python
- FastAPI
- MQTT
- MAVLink

Frontend

- React
- NextJS

AI

- llama.cpp
- OpenCV
- YOLO

Robotics

- ROS2
- ArduPilot

Startup Flow

1. detect hardware

2. initialize plugins
3. start telemetry
4. connect dashboard
5. wait for mission prompt
6. generate mission
7. execute mission
8. save logs and memory

BOOTFLOW.md

Nanggrove OS AI — Boot Flow

Stage 1 — Initialization

- load configs
- initialize logger
- initialize memory
- initialize plugin manager

Stage 2 — Hardware Discovery

- scan USB
- scan serial
- scan network
- probe telemetry
- enumerate sensors

Stage 3 — Agent Startup

- start Hermes
- start PicoClaw
- load memory context

Stage 4 — Dashboard Startup

- initialize API
- initialize websocket
- initialize map layer

Stage 5 — Mission Ready

- wait for prompt
- wait for command
- wait for hardware assignment

PLUGIN_GUIDE.md

Nanggrove OS AI — Plugin Guide

Plugin Purpose

Plugins extend Nanggrove OS AI into new hardware domains.

Plugin Requirements

Every plugin should expose:

- metadata
- supported devices
- supported commands
- health checks
- telemetry hooks
- calibration hooks

Example Plugin

Air Plugin

Capabilities:

- flight telemetry
- waypoint execution
- payload drop
- return-home
- mapping capture

Plugin Folder

Each plugin should contain:

- manifest
- adapter
- commands
- telemetry parser
- optional UI widgets

DEV_GUIDE.md

Nanggrove OS AI — Developer Guide

Development Principles

- modular first
- deterministic control
- logs everywhere
- simple before complex
- reusable interfaces

Coding Style

- explicit names
- low hidden state
- no giant files
- clear adapters
- versioned configs

Recommended Workflow

1. create issue
2. create module
3. create tests
4. validate logs
5. connect dashboard
6. document behavior

Warning

Do not tightly couple AI reasoning with realtime hardware execution.

DEPLOYMENT.md

Nangroe OS AI — Deployment

Local Development

- Linux preferred
- Docker optional
- SQLite default

Edge Device

- Raspberry Pi
- Jetson Nano
- Mini PC

Drone Companion Computer

- Raspberry Pi 5
- telemetry bridge
- camera processing
- mission sync

Cloud Sync

Optional:

- telemetry mirror
- project backup
- fleet analytics

SECURITY_BASELINE.md

Nangroe OS AI — Security Baseline

V1 Goals

- local authentication
- session isolation
- encrypted secrets
- signed configs optional

Not Included in V1

- military-grade anti-clone
- hardware attestation
- swarm cryptography

Rule

Security should not block the first working MVP.

MILESTONES.md

Nangroe OS AI — Milestones

Milestone 1

Core repository and dashboard shell.

Milestone 2

Hardware detection and telemetry.

Milestone 3

Mission engine and Hermes responses.

Milestone 4

Drone mapping workflow.

Milestone 5

Payload automation and voice.

Milestone 6

Cross-domain plugin support.

VISION.md

Nanggroe OS AI — Vision

The long-term vision is to create:

- AI-native robotics orchestration,
- autonomous mission systems,
- prompt-driven hardware workflows,
- reusable robotics infrastructure,
- modular real-world automation.

The system should eventually support:

- drones,
- rovers,
- marine vehicles,
- agriculture,
- factories,
- mapping,
- monitoring,
- logistics.

MISSION_PIPELINE.md

Nanggroe OS AI — Mission Pipeline

Flow

Prompt → Intent Parsing → Mission Generation → Hardware Validation → Telemetry Monitoring
→ Execution → Logging → Memory Sync

Example

User:

"Map this field and estimate area."

System:

1. detect mapping-capable hardware
2. validate GPS and camera
3. generate survey grid
4. execute mission
5. store images
6. calculate area
7. generate summary

Sudah rapi menjadi repo-ready pack lengkap:

- struktur repository penuh,
- backend/frontend layout,
- plugin system,
- boot flow,
- deployment,
- developer guide,
- mission pipeline,
- security baseline,
- milestone roadmap,
- dan seluruh `.md` blueprint siap dipisah ke repo nyata.

Ini level foundation engineering document untuk mulai membangun **Nanggroe OS AI** secara serius dan modular.